Reinforcement Learning
Programming assignment-1
REPORT

Team-04

Team Members:

- K Sai Bharat Kumar Varma - SE22UCSE125
- Dev M Bandhiya - SE22UCSE078
- G Aditya Vardhan - SE22UCSE092
- K Harpith Rao - SE22UCSE141

Contributions:

- **K Sai Bharat Kumar Varma (SE22UCSE125):**
  - **Q5 (Implementation of Policy Gradient Algorithm)**:
    Worked on implementing the policy gradient algorithm for contextual bandits in ServerAllocationEnv. Designed a PyTorch-based neural network with softmax output for action selection and integrated an epsilon-greedy exploration strategy with exponential decay. Used gradient ascent to update the policy, computing loss as the negative log probability of the chosen action weighted by the reward. Ensured immediate updates after each action for dynamic optimization and tracked performance using a receding window time-averaged reward plot.
  - **Q7 (b) & Q7 (c) (Theory on Policy Gradient in Continuous Action Space)**:
    Derived equations for policy gradient in continuous action space using a Gaussian policy distribution, computing gradients for policy optimization. Developed pseudocode for efficient gradient updates, ensuring effective continuous action selection.

- **Dev M Bandhiya (SE22UCSE078):**
  - **Q1 (Theory Question)**:
    Worked on analyzing the sources of noise in rewards, including randomness in processing time, network usage variability, scheduling uncertainty, and random job arrivals

  - **Q3 (Implementation of Greedy Method)**:
    Implemented the ε-greedy algorithm for linear bandits in the ServerAllocationEnv, incorporating an adaptive exploration strategy with exponential decay. Designed parameter updates using least squares

estimation, ensuring efficient learning. Tracked performance by plotting a receding window time-averaged reward to assess policy effectiveness.

- **Q6 (Test Cases for Implemented Methods)**:
  Deployed test cases to analyze whether the number of allocated servers increases as priority increases, the number of allocated servers increases as the estimated processing time increases and the number of allocated servers increases as the number of jobs increases for Greedy policy.

- **G Aditya Vardhan (SE22UCSE092):**
  - **Q7 (a) (Code for sampling from Gaussian Distribution):**
    implemented a Python function to sample actions from a Gaussian distribution, computing the mean and standard deviation using policy parameters and context features. This function ensures continuous action selection in policy gradient methods.

  - **Q6 (Test Cases for Implemented Methods)**:
    Deployed test cases to analyze whether the number of allocated servers increases as priority increases, the number of allocated servers increases as the estimated processing time increases and the number of allocated servers increases as the number of jobs increases for Policy Gradient and Upper Confidence Bound.

- **K Harpith Rao (SE22UCSE141):**
  - **Q2 (Theory Question):**
    Addressed the challenge of varying context sizes in ML/DL models by implementing feature aggregation, job type encoding, and zero-vector handling for empty job slots. Optimized feature representation using normalization to prevent scale imbalance, improve model convergence, and maintain consistency across time slots.

  - **Q4 (Implementation of Upper Confidence Bound):**
    Implemented the Upper Confidence Bound (UCB) algorithm for linear bandits in the ServerAllocationEnv. Designed a decaying epsilon strategy, optimized action selection using confidence bounds, and updated model parameters incrementally to enhance decision-making. Plotted a receding window time-averaged reward to track performance trends.

Q1. The sources of noise in the reward associated with an action are as follows:

- **<u>Randomness in processing time</u>:**
  The actual processing time of the job is not known. Even though we know

the estimated processing time for each job, there is a possibility that it does not match with the actual (true) processing time.

- **Variability in Network Usage:**
  Jobs have different levels of network usage (ρi,tpi,t), which impacts the true processing time and contributes to uncertainty in the system. Moreover, Higher network usage tends to increase processing time but not in a strictly deterministic manner.

- **Scheduling Uncertainty:**
  The highest-priority-first scheduling introduces unpredictability which can lead to different execution time for jobs depending on the current system state. Different servers complete jobs at different rate which creates a variability in the waiting time of remaining jobs.

- **Random arrival of jobs:**
  The number of jobs arriving in a time slot (nt) is a random integer between 1 and 8. This affects the workload and consequently, the server allocation decision.

Q2.

A) How to deal with the fact that the context size is varying? This is important because most ML/DL models can't deal with varying input size.

Answer:

Most ML/DL models require a fixed-size input, but the number of jobs (nt) varies at each time step. We addressed this challenge by implementing the following strategies in the code:

Aggregating features:

Instead of processing individual jobs separately, key metrics are computed at a batch level, such as:

- Average Priority across all jobs.
- Average Network Usage to reflect overall communication load.
- Average Processing Time to capture execution complexity.

Encoding Job Type Distribution:

The relative frequency of job types (A, B, C) is stored as a fixed-size vector rather than absolute counts, ensuring a uniform representation across varying numbers of jobs. The following formula is used:

Relative frequency of a job type(*i)* = count of jobs of type(*i*) / *nt*

where, *nt* is the total number of jobs arriving in the time slot.

Handling Empty Job Slots:
If no jobs arrive (nt=0), the model returns a zero-vector, preventing inconsistencies due to missing inputs.

By following these strategies, we ensured that a consistent format of input to the model regardless of the varying number of arriving jobs.

B) Can we reduce the number of features?

Answer:
Yes, one way to optimize the feature representation is through feature normalization. We have implemented this in our code for the following reasons:

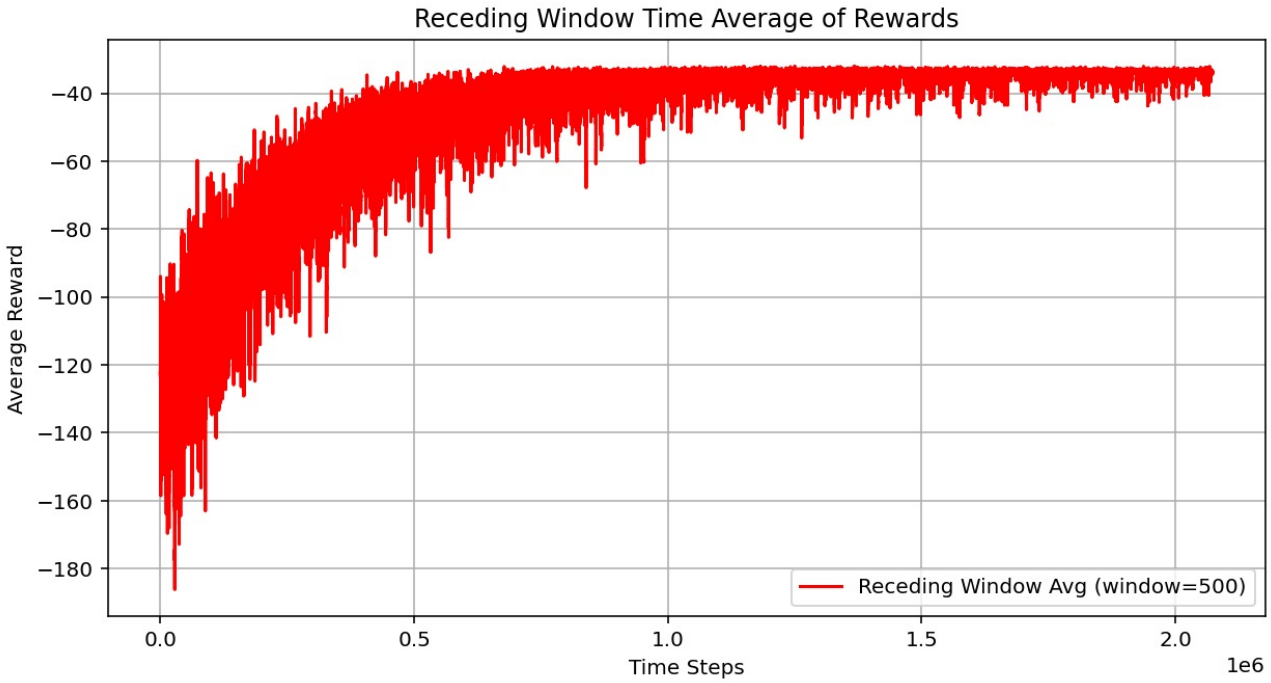
- **Avoid scale imbalance:**
  Job priority, network usage, and processing time may have different numerical ranges. Without normalization, features with larger values (e.g., processing time) may dominate the learning process.

- **Improves Model Convergence**:
  Normalized values help reinforcement learning algorithms learn patterns more effectively.

- **Ensures Consistency Across Different Time Slots**:
  Since the number of jobs varies per time slot, using average values ensures that the feature representation remains stable.

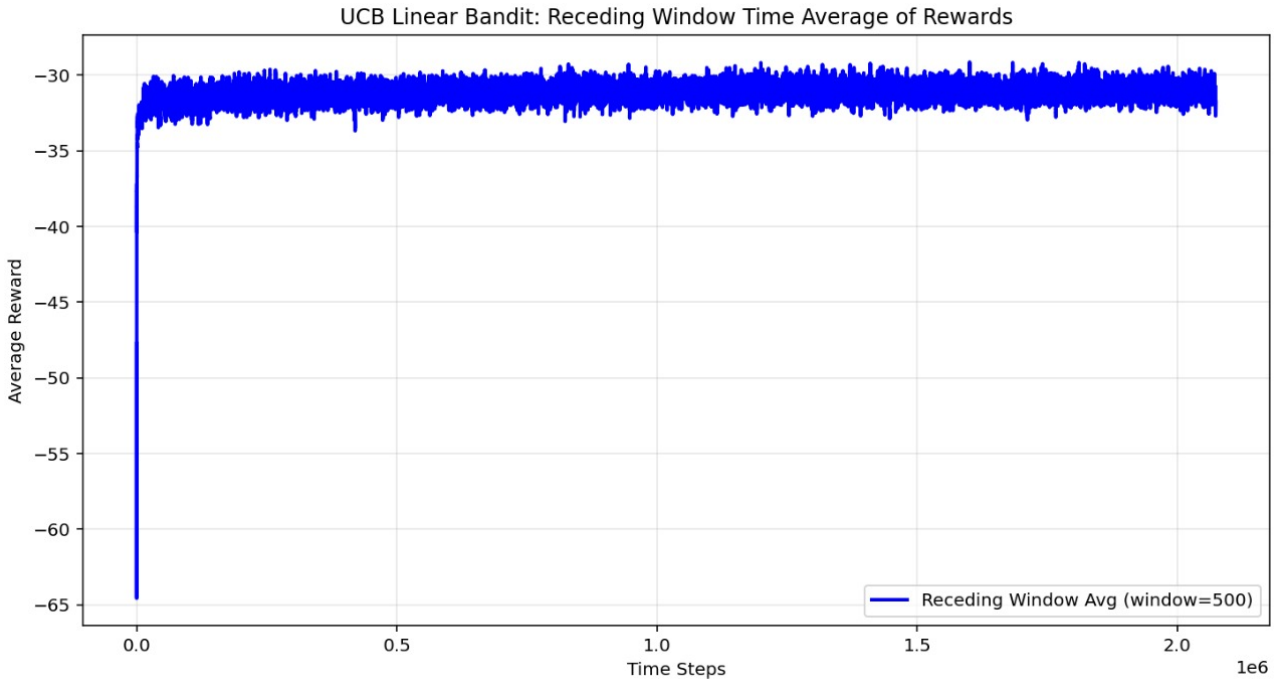
Q3.

Receding window time averaged reward
The average reward for greedy policy: -33.94498043705603



Q4.

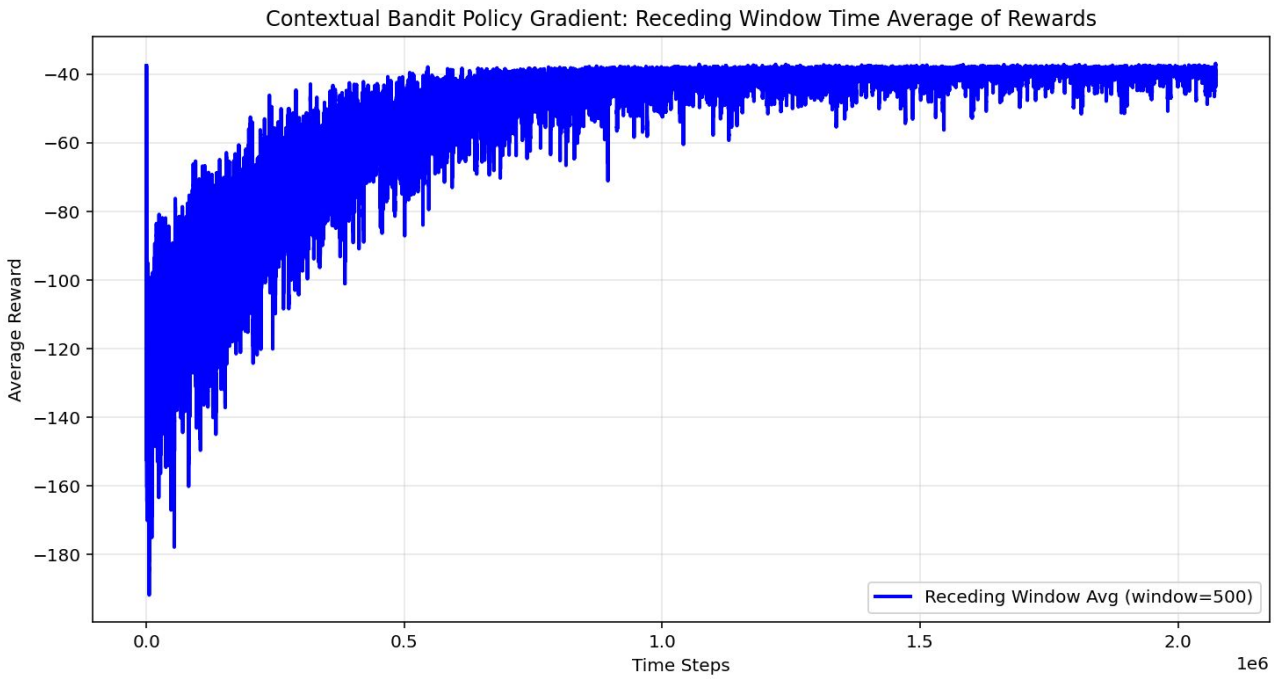
Receding window time averaged reward



The average reward for UCB policy: -29.04037769505338

Q5.

Receding window time averaged reward



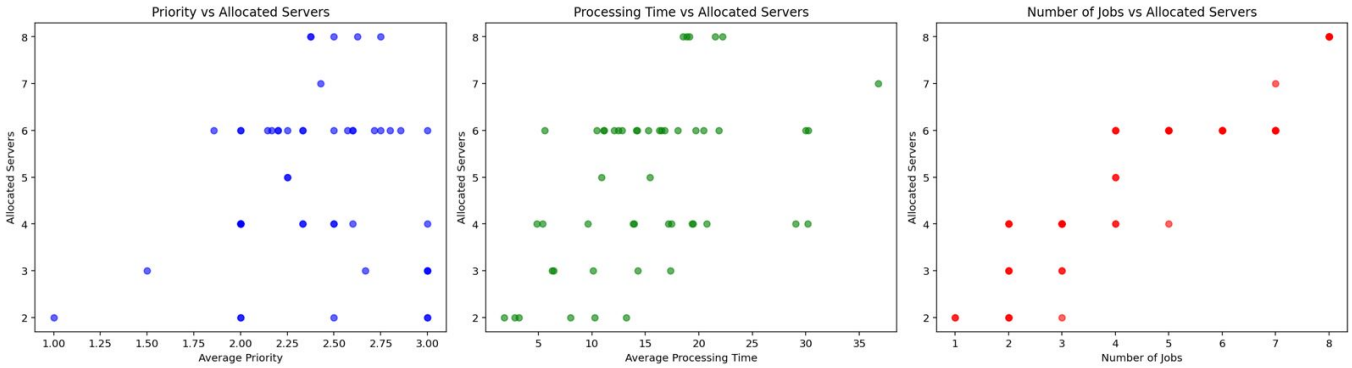
The average reward for Policy Gradient: -37.78663077260764

Q6.

(a) Does the number of allocated servers increase as priority increases?
(b) Does the number of allocated servers increase as the estimated processing time increases?
(c) Does the number of allocated servers increase as the number of jobs increases?

Answer:

For Greedy policy:

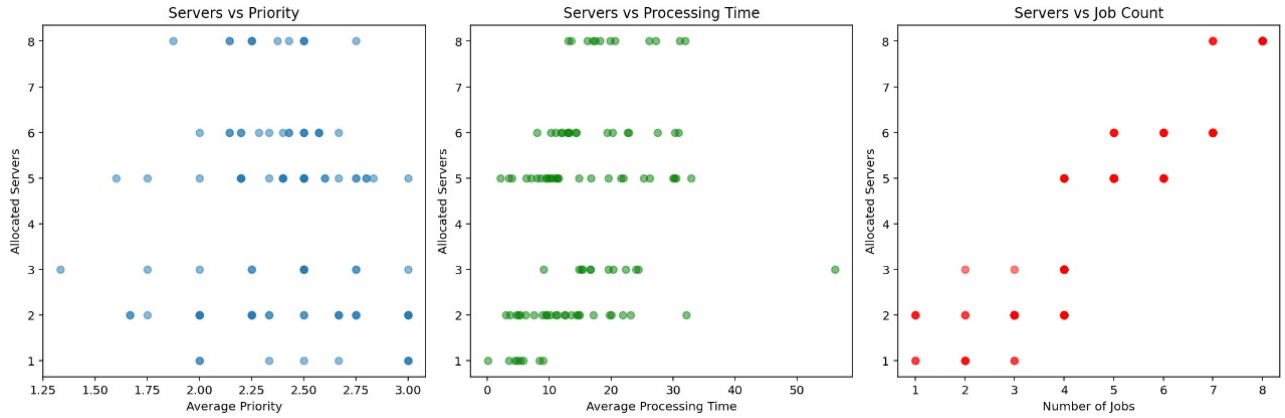


Explanation: The greedy policy always selects the action that has the highest estimated reward at the current time step. It does not explore other actions that might yield a better reward in the future.

Trend Observed in Graphs:

- The allocated servers are widely distributed across different values for priority and processing time.
- The number of allocated servers increases with the number of jobs, showing that the policy responds to workload increases.
- However, since the greedy policy does not explore much, it may get stuck in a suboptimal allocation where certain values dominate.

For UBC policy:

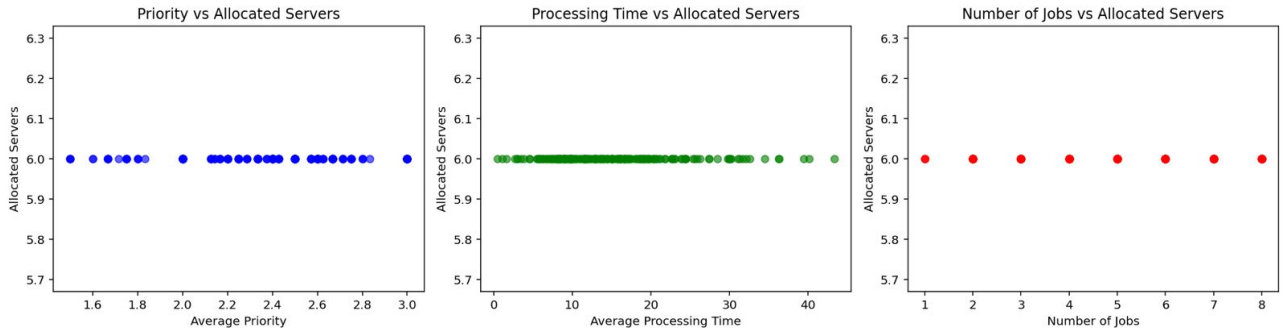


Explanation: The UCB policy balances exploration and exploitation by selecting actions based on both their estimated reward and the uncertainty in those estimates. Actions with higher uncertainty are explored more.

Trend Observed in Graphs:

- The allocated servers remain almost constant at a particular value (close to 6) for all priority, processing time, and job count values.
- This suggests that UCB finds a stable server allocation strategy that does not fluctuate much.
- Unlike Greedy, which might make inconsistent allocations, UCB ensures steady performance but may not always reach optimal efficiency.

For Policy Gradient:



Explanation: Policy gradient methods learn a policy directly by updating the probabilities of selecting actions based on rewards received. These methods continuously adjust actions based on reward feedback.

Trend Observed in Graphs:

- The allocation of servers varies depending on priority, processing time, and job count, showing dynamic adaptation.
- The number of allocated servers increases gradually with the number of jobs, similar to the Greedy policy, but with smoother variations.
- Policy gradient is more adaptive than Greedy and more flexible than UCB, as it learns from the environment while maintaining exploration.

Q7.

(a) Python code to sample an action from a Gaussian distribution of mean and standard deviation

```
def sample_action(theta1, theta2, x):
    mean = np.dot(theta1.T, x)
    std_dev = np.exp(np.dot(theta2.T, x))
    return np.random.normal(mean, std_dev)
```

Here np refers to the python library numpy.

(b)

(1)

(Q 7)

(b) Gaussian distribution with mean $\theta_1^T x$ and standard deviation $e^{\theta_2^T x}$ $\Rightarrow$

$$\pi(a|x,\theta) = \frac{1}{e^{\theta_2^T x}\sqrt{2\pi}} e^{-\frac{(a-\theta_1^T x)^2}{2(e^{\theta_2^T x})^2}} \rightarrow (1)$$

Estimate of $\nabla_\theta J(\theta) \Rightarrow$

$$\nabla_\theta \left(\frac{1}{N} \sum_{t=1}^{N} \gamma_t \log(\pi(a_t|\theta, x_t)) \right)$$

for discrete action space.

for continous action space:-

$$\nabla_\theta J(\theta) = \int \gamma_t \nabla_\theta \log \pi(a|x,\theta)\, \pi(a|x,\theta)\, da \rightarrow (2)$$

$\Rightarrow$ To find $\log(\pi(a|x,\theta)) \Rightarrow$

applying log to both sides to equation (1):-

$$\log(\pi(a|x,\theta)) = \log\left(\frac{1}{e^{\theta_2^T x}\sqrt{2\pi}} e^{-\frac{(a-\theta_1^T x)^2}{2(e^{\theta_2^T x})^2}} \right) \rightarrow (3)$$

(2)

$$\Rightarrow \log(\pi(a|x,\theta)) = -\frac{1}{2}\log(2\pi) - \theta_2^T x - \frac{(a-\theta_1^T x)^2}{2(e^{\theta_2^T x})^2} \rightarrow (4)$$

$\Rightarrow$ To compute $\nabla_{\theta_1} J \Rightarrow$

Differentiate $\log(\pi(a|x,\theta))$ with respect to θ_1.

$$\frac{\partial}{\partial\theta_1}\log(\pi(a|x,\theta)) = \frac{\partial}{\partial\theta_1}\left(-\frac{1}{2}\log(2\pi) - \theta_2^T x - \frac{(a-\theta_1^T x)^2}{2(e^{\theta_2^T x})^2}\right)$$

$$\Rightarrow \frac{\partial}{\partial\theta_1}\left(-\frac{1}{2}\log(2\pi)\right) - \frac{\partial}{\partial\theta_1}\left(\theta_2^T x\right) + \frac{\partial}{\partial\theta_1}\left(-\frac{(a-\theta_1^T x)^2}{2(e^{\theta_2^T x})^2}\right)$$

$$\Rightarrow 0 + 0 + \frac{\partial}{\partial\theta_1}\left(\frac{-(a-\theta_1^T x)^2}{2(e^{\theta_2^T x})^2}\right)$$

$$\Rightarrow -\frac{1}{2} \times \frac{1}{e^{2\theta_2^T x}} \times \frac{\partial}{\partial\theta_1}\left((a-\theta_1^T x)^2\right)$$

$$\Rightarrow \frac{-1}{2e^{2\theta_2^T x}}\left(-2(a-\theta_1^T x)x\right)$$

$$\Rightarrow \frac{(a-\theta_1^T x)}{e^{2\theta_2^T x}}x = \log(\pi(a|x,\theta))$$

Substituting $\log(\pi(a|x,\theta))$ in (2) :-

(3)

$$\nabla_{\theta_1} J = \int \gamma_1 \log(\pi(a|x,\theta))\, \pi(a|x,\theta)\, da$$

$$= \int \gamma_1 \left(\frac{(a-\theta_1^T x)}{e^{2\theta_2^T x}} x \right) \pi(a|x,\theta)\, da$$

$$\Rightarrow E_{\pi_\theta}[(a-\theta_1^T x)] = 0$$

$$E_{\pi_\theta}[(a-\theta_1^T x).\gamma_t] = \gamma_t E_{\pi_\theta}[(a-\theta_1^T x)]$$

$$\nabla_{\theta_1} S_1 = E_{\pi_\theta}\left[\gamma_t \frac{(a-\theta_1^T x)}{e^{2\theta_2^T x}} x \right]$$

where E_{π_θ} is Expectation.

$\Rightarrow$ To compute $\nabla_{\theta_2} S$:-

differentiate (4) with respect to θ_2

$$\frac{\partial}{\partial \theta_2} \log(\pi(a|x,\theta)) = \frac{\partial}{\partial \theta_2} \left(-\frac{1}{2}\log(2\pi) - \theta_2^T x - \frac{(a-\theta_1^T x)^2}{2(e^{2\theta_2^T x})} \right)$$

$$\frac{\partial}{\partial \theta_2} \log(\pi(a|x,\theta)) = \frac{\partial}{\partial \theta_2}\left(-\frac{1}{2}\log(2\pi)\right) - \frac{\partial}{\partial \theta_2}(\theta_2^T x) +$$

$$\frac{\partial}{\partial \theta_2}\left(\frac{-(a-\theta_1^T x)^2}{2 e^{2\theta_2^T x}} \right)$$

(4)

$$\Rightarrow \quad 0 + (-x) + \frac{\partial}{\partial \theta_2}\left(\frac{-(a-\theta_1^T x)^2}{2e^{2\theta_2^T x}}\right)$$

$$\Rightarrow \quad -x + \left(\frac{-(a-\theta_1^T x)^2}{2}\right)\frac{\partial}{\partial \theta_2}\left(\frac{1}{e^{2\theta_2^T x}}\right)$$

$$\Rightarrow \quad -x + \left(\frac{-(a-\theta_1^T x)^2}{2}\right)\left(\frac{-2x}{e^{2\theta_2^T x}}\right)$$

$$\Rightarrow \quad -x + \frac{(a-\theta_1^T x)^2}{e^{2\theta_2^T x}}x$$

$$\Rightarrow \quad \frac{\partial}{\partial \theta_2}\log(\pi(a|x,\theta)) = -x + \frac{(a-\theta_1^T x)^2}{e^{2\theta_2^T x}}x$$

Substituting this in eq (2):-

$$\nabla_{\theta_2} J = \int r_t\left(-x + \frac{(a-\theta_1^T x)^2}{e^{2\theta_2^T x}}x\right)\pi(a|x,\theta)\,da$$

From Gaussian expectation:-

$$E_{\pi_\theta}[(a-\theta_1^T x)^2] = e^{2\theta_2^T x}$$

$$\nabla_{\theta_2} J = E_{\pi_\theta}\left[r_t\left(-x + \frac{(a-\theta_1^T x)^2}{e^{2\theta_2^T x}}x\right)\right]$$

(5)

⇒ Update rule using gradient ascent:-

$$\theta_1^{new} = \theta_1^{old} + \alpha \nabla_{\theta_1} J \qquad \text{where } \alpha = \text{learning rate.}$$

$$\theta_2^{new} = \theta_2^{old} + \alpha \nabla_{\theta_2} J$$

⇒ Substituting the values of $\nabla_{\theta_1} J$ and $\nabla_{\theta_2} J$ ⇒

$$\theta_1^{new} = \theta_1^{old} + \alpha \int \gamma_t \frac{(a - \theta_1^T x)}{e^{2\theta_2^T x}} x \, \pi(a|x,\theta) \, da$$

$$\theta_2^{new} = \theta_2^{old} + \alpha \int \gamma_t \left(-x + \frac{(a - \theta_1^T x)}{e^{2\theta_2^T x}} x \right) \pi(a|x,\theta) \, da$$

Q7.
(c)

(Q7)

(c) Pseudo code of policy gradient for continuous action space.

for episode = 1, ..., max episode
→ observe initial state x_0
for $t = 1, \ldots, T$ (episode length):
→ compute mean $(\mu_t) = \theta_1^T x_t$
→ compute std. deviation $(\sigma_t) = e^{\theta_2^T x_t}$
→ sample action: $a_t \sim N(\mu_t, \sigma_t)$
→ Execute action a_t and recieve reward r_t, next state x_{t+1}.
→ compute log probability of action:-

$$\log \pi(a_t | x_t, \theta) = -\log(\sigma_t \sqrt{2\pi}) - \frac{(a_t - \mu_t)^2}{2\sigma_t^2}$$

→ store (x_t, a_t, r_t)
→ compute returns R_t for each t.

compute policy gradients:

$$\nabla_{\theta_1} J = \int r_t \frac{(a - \theta_1^T x_t)}{e^{2\theta_2^T x_t}} x_t \pi(a | x_t, \theta)\, da.$$

$$\nabla_{\theta_2} J = \int r_t \left(-x + \frac{(a - \theta_1^T x_t)^2}{e^{2\theta_2^T x_t}} x_t\right) \pi(a | x_t, \theta)\, da.$$

→ update policy parameters using gradient ascent:-

$$\theta_1^{new} = \theta_1^{old} + \alpha \nabla_{\theta_1} J$$

$$\theta_2^{new} = \theta_2^{old} + \alpha \nabla_{\theta_2} J.$$

→ End.